CAMPUS EVENT VOLUNTEER MATCHER

A PROJECT REPORT

Submitted by

THIRUMALA SRI P

(2116240701569)

in partial fulfilment of the End Semester Examination for the course

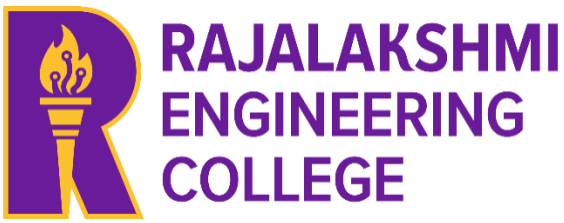
CB23F35 –

LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF ENGINEERING

COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

NOVEMBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled "CAMPUS EVENT VOLUNTEER MATCHER" is the bonafide work of THIRUMALA SRI P (2116240701569) who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator, during the academic year 2026–2027, Semester V, under my supervision.

SIGNATURE

Dr. J. Manoranjini
HEAD OF THE DEPARTMENT
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Thandalam – 602 105

SIGNATURE

Dr B.Bhuvaneswaran
COURSE FACULTY
Assistant Professor
Department of Computer Science and Engineering
Rajalakshmi Engineering College

Submitted for the End Semester Examination (Individual Project) held on –.

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

I, THIRUMALA SRI P (2116240701569), hereby declare that the project report titled "CAMPUS EVENT VOLUNTEER MATCHER", submitted for the End Semester Examination of the course CB23F35 – Low-Code Application Development using Zoho Creator, is a record of original work carried out by me individually. The Zoho Creator application described in this report — including its data model, forms, reports, pages, Deluge scripts, workflows and integrations — was designed and developed by me.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. Every template, sample application, AI-assisted feature (such as Zia or the AI application builder), code snippet or external resource used in this work has been duly acknowledged in the report.

Place: Chennai
Date: –

SIGNATURE OF THE STUDENT
THIRUMALASRI P
2116240701569

ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman Mr. S. Meganathan, Chairperson Dr. (Mrs.) Thangam Meganathan and Vice-Chairman Mr. M. Abhay Shankar for providing the infrastructure and facilities that made this work possible.

I am grateful to our Principal, Dr. S. N. Murugesan, Rajalakshmi Engineering College, for the constant encouragement extended to students to take up industry-relevant learning.

I thank Dr. J. Manoranjini, Head of the Department, Department of Computer Science and Engineering, for the guidance and support provided throughout the course.

I owe my sincere thanks to my course faculty, Dr.B.Bhuvaneswaran, Assistant Professor, for the valuable guidance, reviews and feedback that shaped this application and report.

I also acknowledge Zoho Corporation for providing access to the Zoho Creator platform and learning resources, and I thank my family and friends for their unwavering support.

THIRUMALA SRI P

ABSTRACT

College events such as symposiums, cultural festivals, workshops and sports meets depend on student volunteers for registration desks, hospitality, stage management, technical support and logistics. Managing volunteers manually is time-consuming: organisers collect applications through messages and forms, assign roles by hand, track registrations in separate sheets and cannot easily see how many volunteers each role has or which applications are still pending.

The Campus Event Volunteer Matcher is a web-based application developed on Zoho Creator that provides a centralised platform for volunteer coordination. Its modules are Event Registrations, Volunteer Registrations, Volunteer Requirements, Volunteer Applications and a Dashboard. Volunteer Applications record the applicant, the selected requirement, application date, reason for volunteering, application status and organiser remarks, while Volunteer Requirements define the roles and slots needed for each event. Deluge scripts and workflows validate applications, keep the count of filled slots, notify volunteers of decisions and remind approved volunteers before their event. The dashboard summarises total applications, requirements, volunteer registrations and event registrations, shows the Approved, Rejected, Pending and Completed counts, and charts the distribution of volunteers across roles.

The application was verified through form, workflow and role-based test cases and is accessible on the web and the Zoho Creator mobile app. It reduces manual administrative effort, makes the status of every application transparent to organisers and volunteers, and gives organisers a structured, data-driven way to coordinate volunteers for campus events.

Keywords: *Volunteer Management; Campus Event Management; Zoho Creator; Application Tracking; Volunteer Registration; Dashboard Analytics; Workflow Management*

PROJECT AT A GLANCE

Attribute	Details
Project Title	Campus Event Volunteer Matcher
Application Domain	Education – campus event and volunteer management
Student Name & Register No.	THIRUMALA SRI (2116240701569)
Zoho Creator Edition	Zoho Creator (free trial edition, India data centre)
Number of Forms / Subforms	4 / 0
Number of Reports (by type)	List: 5 Kanban: 0 Calendar: 0 Pivot/Chart: 2 Others: 0
Number of Pages / Dashboards	1 (Dashboard)
Number of Workflows	Form: 6 Schedule: 1 Approval: 0 Blueprint: 0 Batch: 0
Deluge Functions / Custom Actions	8 Deluge scripts including 1 function (notifyVolunteer)
Integrations	Zoho Mail (sendmail task)
Roles & Profiles Defined	Administrator, Event Organizer, Student Volunteer, Faculty Advisor
Environments Used	Single application workspace; separate Stage and Production environments were not configured
Date of Production Deployment	Not separately deployed; live application URL used for demonstration
Source Code Repository	https://github.com/thirumalasrip240701569/LOW-CODE-APPLICATION-USING-ZOHO-CREATOR.git

ABSTRACT iv
PROJECT AT A GLANCE v
LIST OF TABLES ix
LIST OF FIGURES x
LIST OF ABBREVIATIONS x
INTRODUCTION 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 2
1.4 Scope of the Project 2
1.5 Justification for a Low-Code Approach 2
1.6 Organisation of the Report 3
LITERATURE SURVEY AND SYSTEM STUDY 4
2.1 Study of the Existing System 4
2.2 Limitations of the Existing System 4
2.3 Low-Code versus Traditional Development 5
2.4 Literature Survey 6
2.5 Emerging Trends in Low-Code Development 6
2.6 Proposed System 7
REQUIREMENT ANALYSIS 6
3.1 Stakeholders and User Roles 6
3.2 Functional Requirements 6
3.3 Non-Functional Requirements 7
3.4 Hardware and Software Requirements 10
3.5 Use Case Model 10
SYSTEM DESIGN 8
4.1 System Architecture 8
4.2 Data Model 9
4.3 Form Design 9
4.4 Process and Workflow Design 10
4.5 User Interface and Navigation Design 11
4.6 Access Control Design 11
IMPLEMENTATION 12
5.1 Application Setup 12
5.2 Forms and Data Capture 12
5.3 Reports 13
5.4 Pages and Dashboards 14

5.5 Deluge Scripting 15
5.6 Workflows and Automation 18
5.7 Integrations 21
5.8 User Management and Role-Based Access 22
5.9 AI and Analytics Features 28
5.10 Pro-Code Extension 28
5.11 Mobile and Portal Access 28
TESTING AND DEBUGGING 24
6.1 Testing Strategy 24
6.2 Test Cases 24
6.3 Debugging Deluge Scripts 25
6.4 Defect Log 26
6.5 User Acceptance Testing 32
DEPLOYMENT, SECURITY AND GOVERNANCE 27
7.1 Environments and Deployment Strategy 27
7.2 Performance Optimisation 34
7.3 Security Features 34
7.4 Governance and Administration 34
RESULTS AND DISCUSSION 29
8.1 Application Output 29
8.2 Achievement of Objectives 29
8.3 Comparison with the Existing System 29
8.4 Limitations of the Low-Code Approach 30
8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum 38
CONCLUSION AND FUTURE ENHANCEMENTS 31
9.1 Conclusion 31
9.2 Future Enhancements 31
9.3 Learning Outcomes and Career Relevance 39
REFERENCES 32

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application 5
Table 2.2: Summary of Literature Survey 6
Table 3.1: Stakeholders and User Roles 6
Table 3.2: Functional Requirements 6
Table 3.3: Non-Functional Requirements 7
Table 3.4: Hardware and Software Requirements 10
Table 3.5: Use Case Description – Apply for a Volunteer Role 11
Table 3.6: Use Case Description – Review a Volunteer Application 11
Table 4.1: Entities and Relationships 9
Table 4.2: Field Specification – Event Registration Form 15
Table 4.3: Field Specification – Volunteer Registration Form 15
Table 4.4: Field Specification – Volunteer Requirement Form 16
Table 4.5: Field Specification – Volunteer Application Form 16
Table 4.6: Role–Permission Matrix 11
Table 5.1: Reports Implemented 13
Table 5.2: Deluge Scripts Implemented 15
Table 5.3: Workflows Implemented 18
Table 5.4: Integrations Implemented 21
Table 6.1: Test Cases 24
Table 6.2: Defect Log 26
Table 6.3: User Acceptance Feedback 32
Table 7.1: Deployment Checklist 27
Table 7.2: Performance Optimisation Results 34
Table 7.3: Security Controls Implemented 28
Table 8.1: Mapping of Objectives to Outcomes 29
Table 8.2: Existing System versus Proposed Application 29
Table A.1: Feature Coverage Checklist 40
Table B.1: Mapping of Course Outcomes to Report Chapters 35
Table E.1: Evaluation Rubric for the Individual Project 39
Table E.2: Performance Level Descriptors 40
Table F.1: Viva-Voce Questions and Responses 41
Table F.2: Marks Summary 41

Figure 2.1: As-Is Process Flow of Manual Volunteer Coordination 4
Figure 3.1: Use Case Diagram of the Campus Event Volunteer Matcher 11
Figure 4.1: System Architecture of the Campus Event Volunteer Matcher 8
Figure 4.2: Entity–Relationship Diagram 9
Figure 4.3: Volunteer Application Process Flow 10
Figure 4.4: Navigation Structure and Wireframe of the Dashboard 11
Figure 5.1: Application Home and Navigation Sections 12
Figure 5.2: Volunteer Application Form in the Form Builder 13
Figure 5.3: All Volunteer Applications Report 14
Figure 5.4: Dashboard Page 15
Figure 5.5: Form Workflows and Reminder Schedule Configuration 21
Figure 5.6: Roles and Permission Profiles Configuration 23
Figure 5.7: Application on the Zoho Creator Mobile App 23
Figure 6.1: Deluge info Output while Debugging the Slot Recount 32
Figure 7.1: Live Application Accessed through the Published URL 27

LIST OF ABBREVIATIONS

Abbreviation	Expansion
CRUD	Create, Read, Update, Delete
ER	Entity–Relationship
FR	Functional Requirement
KPI	Key Performance Indicator
LCAP	Low-Code Application Platform
NFR	Non-Functional Requirement
RBAC	Role-Based Access Control
REST	Representational State Transfer
UAT	User Acceptance Testing
UI	User Interface
Zia	Zoho’s AI assistant

CHAPTER 1
INTRODUCTION

1.1 Background

Engineering colleges host many events every semester — national-level symposiums, cultural festivals, hackathons, workshops, sports meets and orientation programmes — organised by departments, clubs and student bodies. Each event needs student volunteers for roles such as registration desk, hospitality, stage and anchoring, technical support, photography and logistics, and a large event may need dozens of volunteers across several roles.

Today, organisers usually post a call for volunteers in messaging groups, collect replies or Google Form responses, copy them into spreadsheets, select volunteers by hand and inform them individually. The same students may apply several times, some roles are over-subscribed while others stay empty, and after the event there is no consolidated record of who volunteered and in which role. The Campus Event Volunteer Matcher brings events, volunteer requirements, volunteer registrations and applications into one Zoho Creator application with a dashboard for organisers.

1.2 Problem Statement

Volunteer recruitment and coordination for campus events are handled manually through messages, forms and spreadsheets, with no central record of events, roles, volunteers and applications. As a result, organisers spend significant time sorting applications, roles are filled unevenly, and neither organisers nor volunteers can see the current status of an application. The specific pain points are:

- **Scattered applications:** responses arrive through messages and forms and must be copied and de-duplicated by hand.
- **Uneven role allocation:** there is no live count of how many volunteers each role has, so some roles are over-filled and others are short.
- **No status transparency:** volunteers do not know whether their application is pending, approved or rejected, and organisers' remarks are not recorded.
- **Manual communication:** decisions and reminders are sent one by one, and volunteers sometimes forget their reporting time.

- **No consolidated records or reports:** after the event there is no summary of volunteers, roles and participation.

1.3 Objectives

- To design a normalised data model and forms for event registrations, volunteer registrations, volunteer requirements and volunteer applications.
- To automate application validation, slot counting, status notifications, event-completion updates and reminders using Deluge scripting and workflows.
- To provide a role-based dashboard and reports showing totals, application status counts and the distribution of volunteers across roles.
- To integrate the application with Zoho Mail for automated e-mails to volunteers.
- To deploy the application securely to a production environment with governance controls.

1.4 Scope of the Project

In scope: four roles — Administrator, Event Organizer, Student Volunteer and Faculty Advisor — and five modules: Event Registrations, Volunteer Registrations, Volunteer Requirements, Volunteer Applications and the Dashboard. The application covers event and role definition, volunteer registration, applications with status and remarks, automatic slot counting, e-mail notifications, a daily reminder, status-wise and role-wise dashboard analytics, and access on the web and the Zoho Creator mobile app.

Out of scope: participant (attendee) ticketing and payments, certificate generation, QR-based attendance, automatic matching of volunteers to roles by skill, and integration with the college ERP. These are listed as future enhancements in Chapter 9.

1.5 Justification for a Low-Code Approach

A traditional implementation would require a database, a backend with APIs, a web front end, a mobile app, notification and scheduling services and role-based authentication — several weeks of work for a small event-management need. Low-code application platforms provide these as configurable building blocks [1].

Zoho Creator fits this problem well: forms create the data model with lookups between events, requirements, volunteers and applications; list and chart reports and the Page builder give the dashboard; Deluge expresses the validation and slot-count rules; and

schedules and the sendmail task handle reminders and notifications [2]. Hosting, permissions and a mobile app are included, and student organisers can adapt roles, statuses and fields for each new event without a development team [3].

1.6 Organisation of the Report

Chapter 2 describes the existing and proposed systems. Chapter 3 presents the requirements. Chapter 4 explains the system design. Chapter 5 documents the Zoho Creator implementation. Chapter 6 presents testing, Chapter 7 covers deployment and security, Chapter 8 discusses results, and Chapter 9 concludes the report.

CHAPTER 2
EXISTING AND PROPOSED SYSTEM

2.1 Study of the Existing System

In the existing system, volunteers for a campus event are managed as follows:

- A club or department plans an event, and the organiser lists the volunteer roles needed in a note or spreadsheet.
- A call for volunteers is posted in messaging groups; students reply by message or through a Google Form.
- The organiser copies the responses into a sheet and checks duplicates by hand.
- Volunteers are selected and informed individually by phone or message.
- Role-wise counts and attendance are tracked manually on the event day, and no consolidated record remains afterwards.

Figure 2.1 shows this as-is process flow.

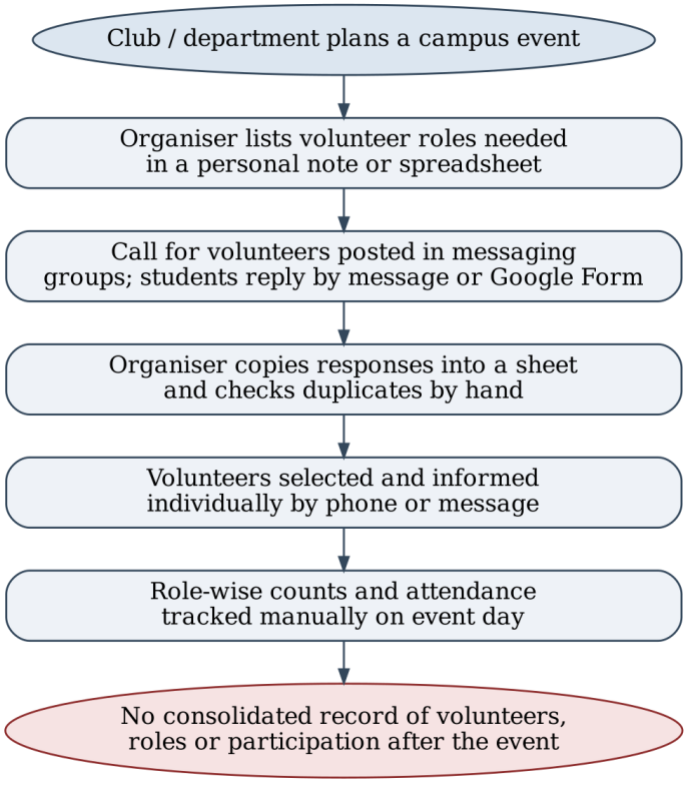


Figure 2.1: As-Is Process Flow of Manual Volunteer Coordination

2.2 Limitations of the Existing System

- Event, role, volunteer and application data is spread across messages, forms and spreadsheets.

- Duplicate applications and over-filled or empty roles are detected only by manual checking.
- Application status and organiser remarks are not recorded, so volunteers have no visibility of decisions.
- Decisions and reminders are communicated one by one, which is slow and error-prone.
- There is no dashboard or report of volunteers by event, role or status.

2.3 Proposed System

The proposed system is a Zoho Creator application with four forms, six form workflows, a daily reminder schedule and a dashboard. It overcomes each limitation of Section 2.2 as follows:

- Events, roles, volunteers and applications are stored once and linked through lookups.
- DS-01 rejects duplicate applications and applications for full or past roles; DS-04 keeps the filled-slot count current.
- Every application has a status (Pending, Approved, Rejected, Completed) and organiser remarks, visible to the volunteer.
- Decisions and reminders are e-mailed automatically.
- The dashboard shows totals, status counts and the role-wise distribution of volunteers.

CHAPTER 3
REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

Table 3.1: Stakeholders and User Roles

Role	Description	Key Responsibilities in the Application
Administrator	Owner of the application (student coordinator / staff)	Manage users and roles, maintain events, view all data and the dashboard
Event Organizer	Student or staff organiser of an event	Register events, post volunteer requirements, review applications, set status and remarks
Student Volunteer	Student who wants to volunteer	Register as a volunteer, apply for roles, track own applications
Faculty Advisor	Faculty member overseeing student events	View the dashboard and reports

3.2 Functional Requirements

Table 3.2: Functional Requirements

FR ID	Requirement	Role(s)	Priority
FR-01	The system shall allow events to be registered with name, type, organising club, date, venue, organiser and status.	Administrator, Organizer	High
FR-02	The system shall allow students to register as volunteers with name, register number, department, year, college e-mail, phone, skills and availability.	Student Volunteer	High
FR-03	The system shall allow volunteer requirements to be defined for an event with role, description, skills, slots required and reporting time.	Organizer	High
FR-04	The system shall allow volunteers to apply for a requirement with a reason for volunteering.	Student Volunteer	High
FR-05	The system shall reject duplicate applications and applications for full roles or past events.	System	High
FR-06	The system shall set new applications to Pending and e-mail an acknowledgement.	System	High
FR-07	The system shall allow organisers to set the application status (Approved, Rejected, Completed) with remarks and notify the volunteer.	Organizer	High
FR-08	The system shall maintain the number of filled slots and mark a requirement Filled when all slots are taken.	System	High

FR ID	Requirement	Role(s)	Priority
FR-09	The system shall mark approved applications Completed when the event is completed.	System	Medium
FR-10	The system shall e-mail approved volunteers a reminder the day before their event.	System	Medium
FR-11	The system shall display a dashboard of totals, status counts and volunteer distribution by role.	Administrator, Organizer, Faculty Advisor	High
FR-12	The system shall restrict each role to permitted forms and reports; volunteers shall see only their own applications.	All	High

3.3 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

NFR ID	Category	Requirement
NFR-01	Performance	Reports with up to 1,000 applications shall load within 3 seconds.
NFR-02	Security	Volunteers shall see only their own registration and applications.
NFR-03	Usability	Applying for a role shall take under a minute, on web or the Zoho Creator mobile app.
NFR-04	Availability	The application shall be available 24 × 7 on Zoho's cloud, subject to Zoho's availability.
NFR-05	Maintainability	New events, roles and statuses shall be addable without code changes.
NFR-06	Auditability	Changes to application status shall be traceable through the audit trail.

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

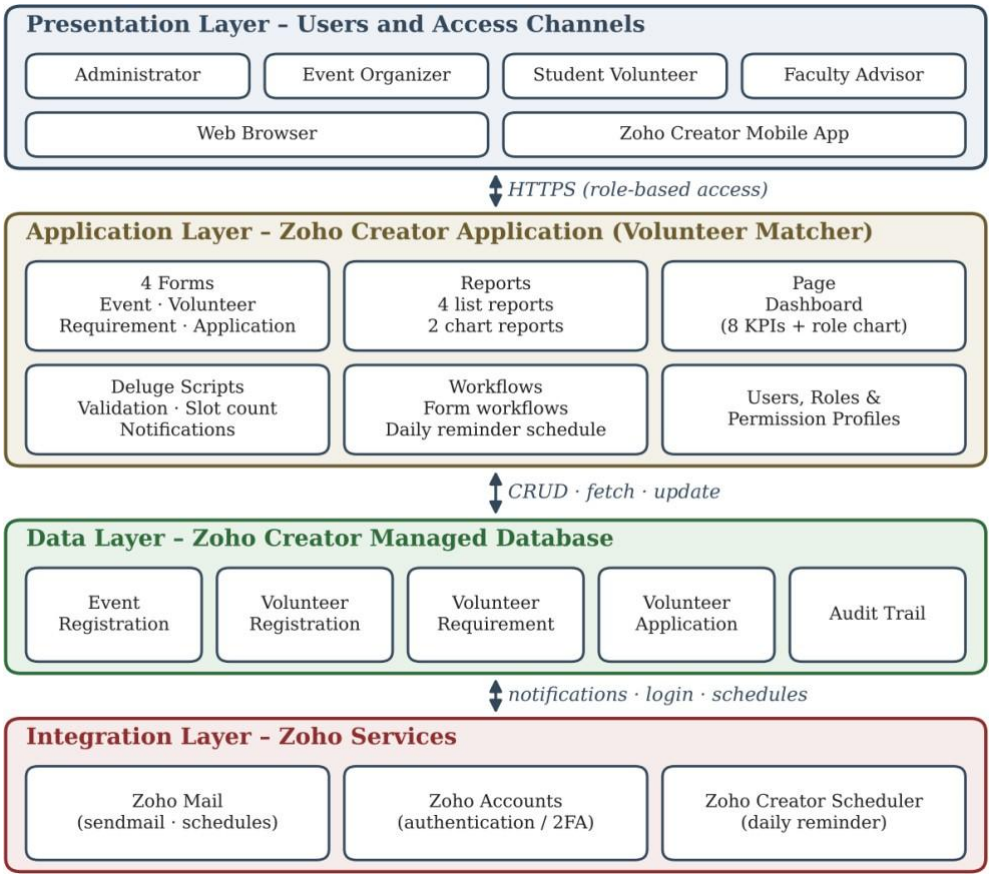


Figure 4.1: System Architecture of the Campus Event Volunteer Matcher

Figure 4.1 shows the four-layer architecture. In the **presentation layer**, the four roles use the application through a web browser or the Zoho Creator mobile app over HTTPS. The **application layer** contains the four forms, list and chart reports, the dashboard page, Deluge scripts, form workflows and the daily schedule, and the roles and permission profiles that control access.

The **data layer** is Zoho Creator's managed database, in which each form is a table linked by lookups, with an audit trail of changes. The **integration layer** contains Zoho Mail for e-mails, Zoho Accounts for authentication and the Creator scheduler for the daily reminder.

4.2 Data Model

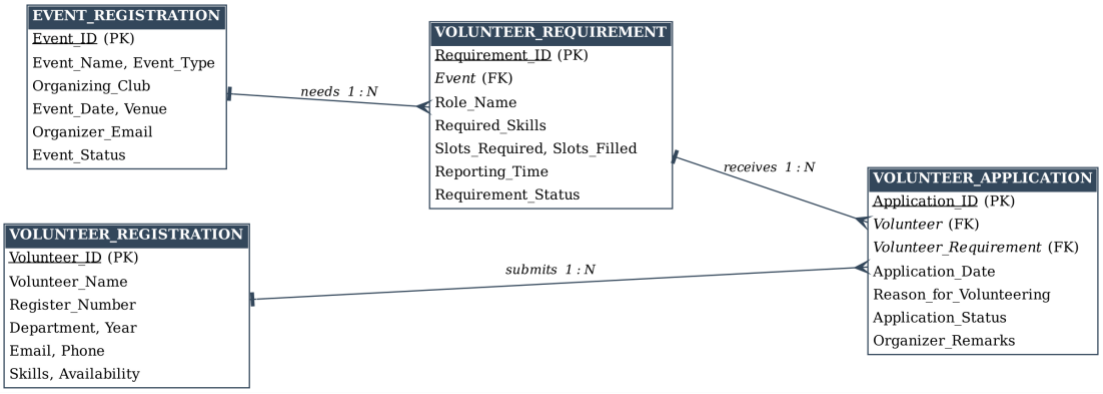


Figure 4.2: Entity–Relationship Diagram

Table 4.1: Entities and Relationships

Entity (Form)	Key Fields	Related To	Relationship Type	Implemented Using
Event Registration	Event_ID, Event_Name, Event_Date	Volunteer Requirement	One-to-Many	Lookup (Event) in Volunteer Requirement
Volunteer Requirement	Requirement_ID, Role_Name, Slots_Required	Volunteer Application	One-to-Many	Lookup (Volunteer_Requirement) in Volunteer Application
Volunteer Registration	Volunteer_ID, Register_Number, Email	Volunteer Application	One-to-Many	Lookup (Volunteer) in Volunteer Application

Event Registration and Volunteer Registration are the master entities. Each event has one or more volunteer requirements (roles), and each application links one volunteer to one requirement. Through these two lookups, an application also identifies its event, which lets reports show volunteers by event, role and status. No subform was needed because each application is a single decision about one role.

4.3 Form Design

Four forms capture event details, volunteer profiles, role requirements and volunteer applications. Lookup fields connect related records, while mandatory fields and validation rules maintain data accuracy.

4.4 Process and Workflow Design

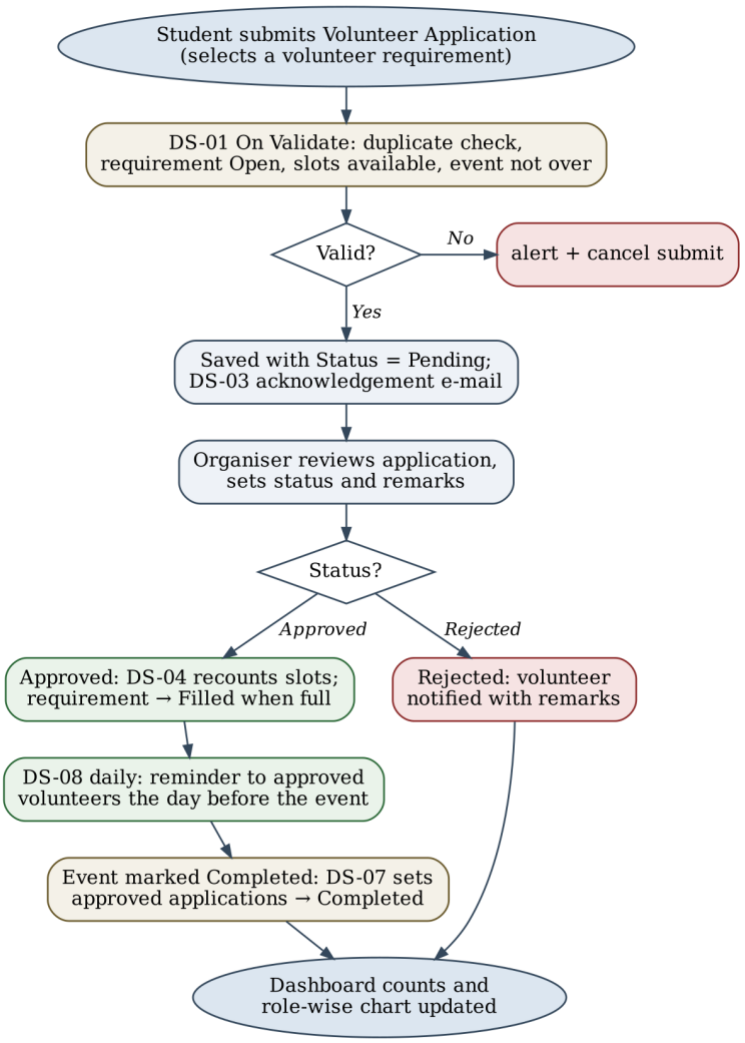


Figure 4.3: Volunteer Application Process Flow

Figure 4.3 shows the main process. A student submits an application for a volunteer requirement; DS-01 rejects duplicates, full roles and past events, and valid applications are saved as Pending with an acknowledgement e-mail (DS-03). The organiser reviews the application and sets it to Approved or Rejected with remarks. On every change, DS-04 recounts the filled slots, marks the requirement Filled when full and e-mails the volunteer. The day before the event, DS-08 reminds approved volunteers, and when the event is marked Completed, DS-07 marks their applications Completed. The dashboard reflects every step.

4.5 User Interface and Navigation Design

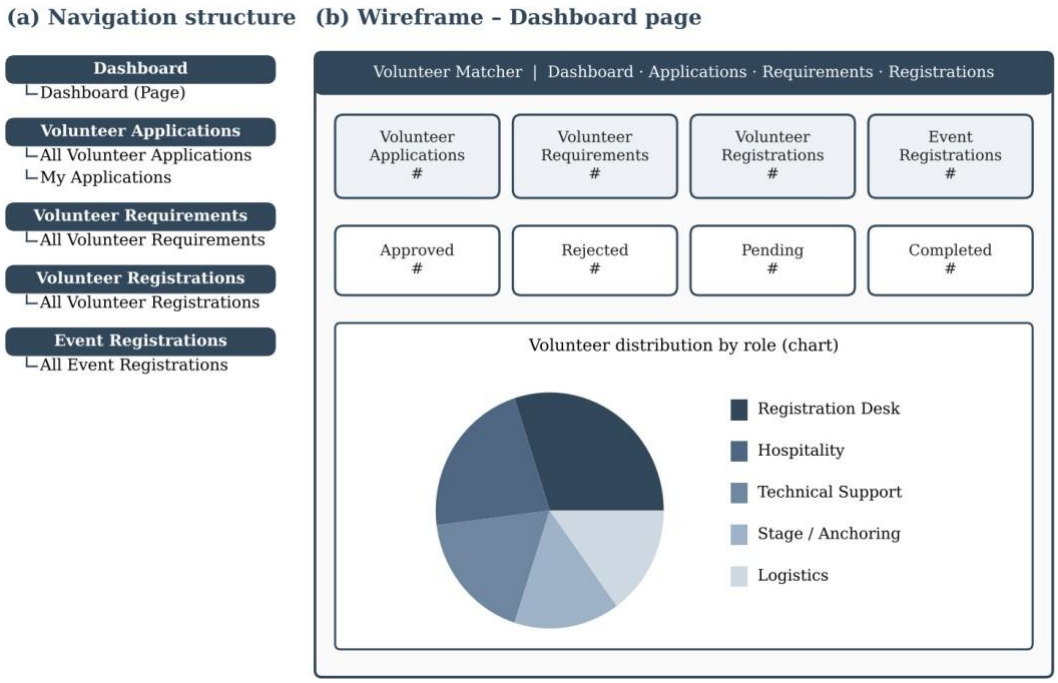


Figure 4.4: Navigation Structure and Wireframe of the Dashboard

The menu follows the application's modules — Dashboard, Volunteer Applications, Volunteer Requirements, Volunteer Registrations and Event Registrations (Figure 4.4(a)). The dashboard (Figure 4.4(b)) shows four total counts, four status counts and a chart of volunteer distribution across roles.

4.6 Access Control Design

Table 4.6: Role–Permission Matrix

Component	Administrator	Event Organizer	Student Volunteer	Faculty Advisor
Form: Event Registration	A E V D	A E V	V	V
Form: Volunteer Registration	A E V D	V	A E V (own)	V
Form: Volunteer Requirement	A E V D	A E V	V (Open only)	V
Form: Volunteer Application	A E V D	E V (status, remarks)	A V (own)	V
Report: My Applications	V	—	V (own)	—
Page: Dashboard	V	V	—	V
Schedules and settings	Full	—	—	—

CHAPTER 5
IMPLEMENTATION

5.1 Application Setup

The application was created in a Zoho Creator workspace (India data centre) and named Campus Event Volunteer Matcher. Event Registration and Volunteer Registration were created first, then Volunteer Requirement (looking up events) and Volunteer Application (looking up volunteers and requirements). Sample events, roles, volunteers and applications were added for testing and demonstration.

The menu presents the Dashboard first, followed by Volunteer Applications, Volunteer Requirements, Volunteer Registrations and Event Registrations, matching the five modules of the application.

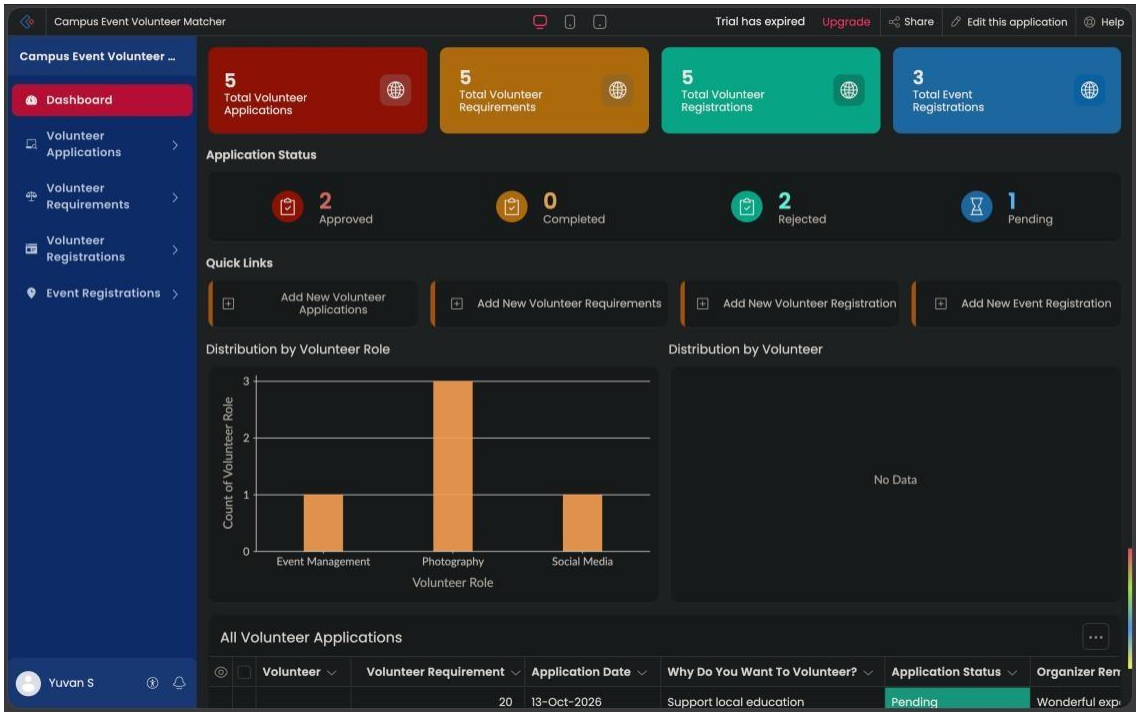


Figure 5.1: Application Home and Navigation Sections

5.2 Forms and Data Capture

The four forms were built with the fields in Tables 4.2–4.5. Key configuration choices were:

- **Lookups:** Event in Volunteer Requirement; Volunteer and Volunteer Requirement in Volunteer Application. The requirement lookup shows only Open requirements.

- **Field validations:** unique register number and e-mail, college e-mail domain (DS-06), at least one slot per requirement (DS-05), and mandatory reason for volunteering.
- **Defaults:** Application Status defaults to Pending and Application Date to today; Slots Filled is read-only and maintained by DS-04.
- **Form properties:** success messages such as "Your application has been submitted and is pending review", and Submit and Reset buttons.

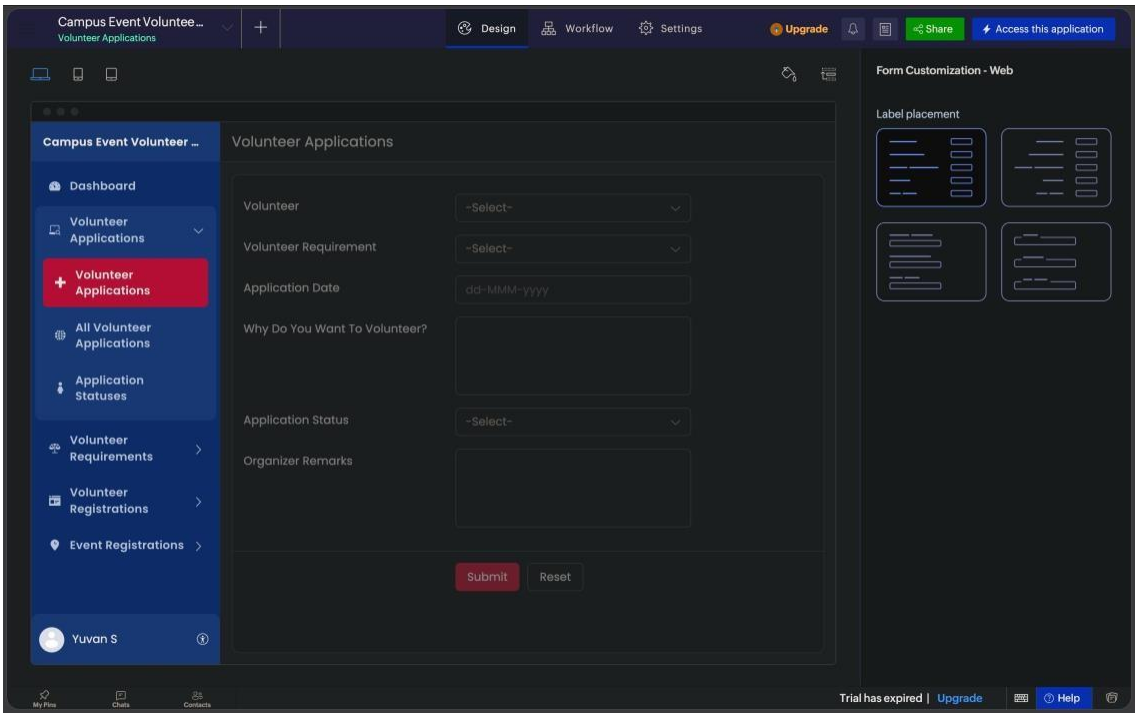


Figure 5.2: Volunteer Application Form in the Form Builder

5.3 Reports

Each form has a list report, students have a My Applications report, and two charts support the dashboard (Table 5.1).

Table 5.1: Reports Implemented

Report Name	Type	Source Form	Filter / Grouping / Sorting	Custom Actions	Roles
All Volunteer Applications	List	Volunteer Application	Group by Application Status; filter by event and role	—	Admin, Organizer

Report Name	Type	Source Form	Filter / Grouping / Sorting	Custom Actions	Roles
My Applications	List	Volunteer Application	Criteria: Volunteer.Email == zoho.loginuserid	—	Student Volunteer
All Volunteer Requirements	List	Volunteer Requirement	Group by Event; show slots required and filled	—	All roles
All Volunteer Registrations	List	Volunteer Registration	Group by Department; search by Skills	—	Admin, Organizer
All Event Registrations	List	Event Registration	Sort by Event Date	—	All roles
Applications by Status	Chart	Volunteer Application	Count by Application Status	—	Dashboard
Volunteers by Role	Chart	Volunteer Application	Count of approved applications by requirement role	—	Dashboard

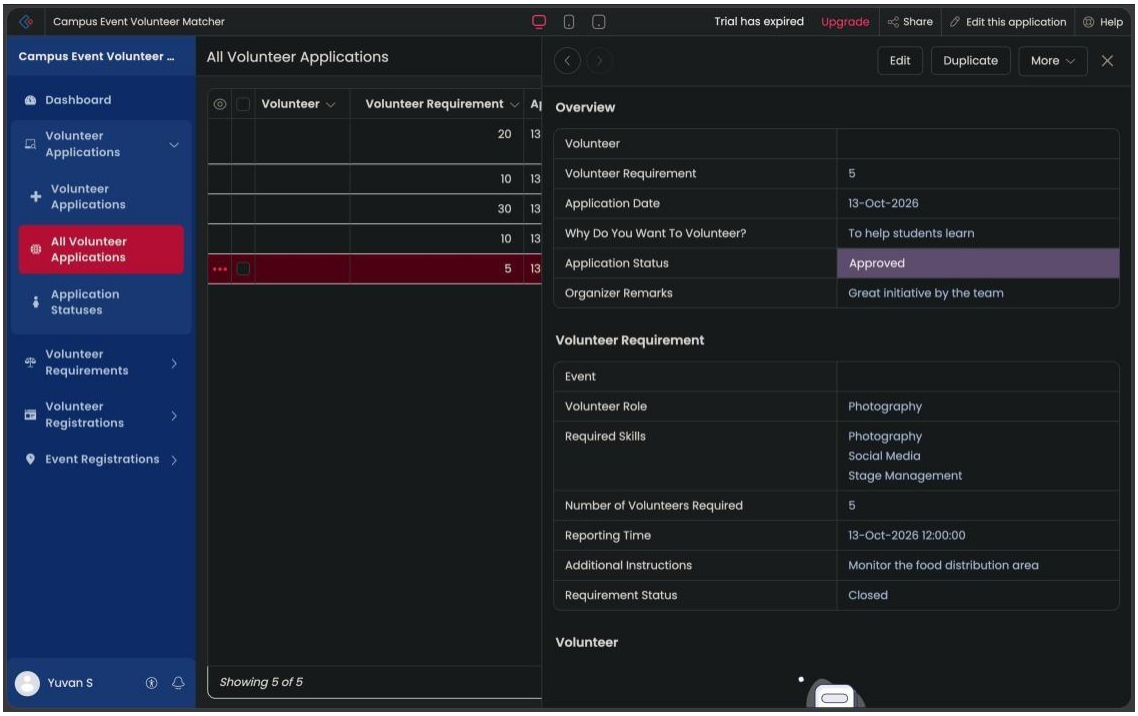


Figure 5.3: All Volunteer Applications Report

5.4 Pages and Dashboards

The **Dashboard** page was built with the Page builder for the Administrator, Event Organizer and Faculty Advisor. It contains:

- **Four total panels:** Volunteer Applications, Volunteer Requirements, Volunteer Registrations and Event Registrations.
- **Four status panels:** Approved, Rejected, Pending and Completed applications.
- **A chart of volunteer distribution across roles**, which shows organisers how volunteers are allocated.

All panels read live data, so the dashboard always reflects the latest applications and decisions.

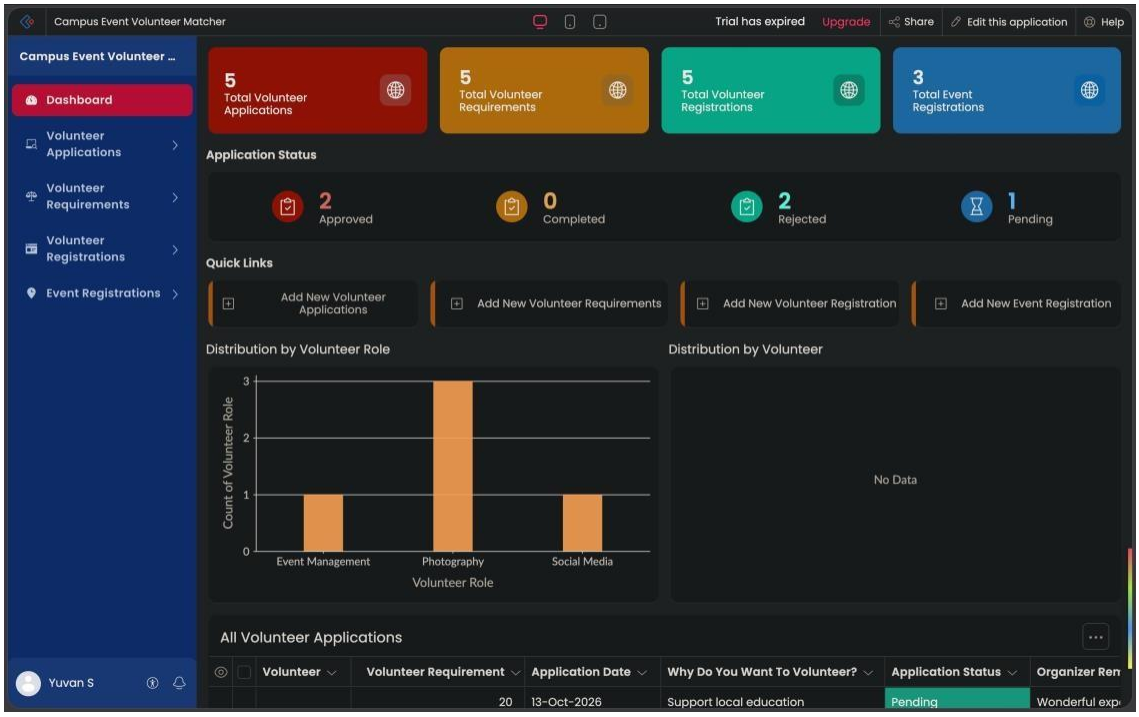


Figure 5.4: Dashboard Page

5.5 Deluge Scripting

Table 5.2: Deluge Scripts Implemented

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-01	Volunteer Application → Validations on form submission (Created)	Reject duplicates, full roles and past events; set Pending	fetch, count(), date comparison, alert, cancel submit
DS-02	Function: notifyVolunteer	Common e-mail to a volunteer	function with arguments, sendmail
DS-03	Volunteer Application → Successful submission (Created)	Acknowledge the application	fetch, function call

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-04	Volunteer Application → Successful submission (Edited)	Recount filled slots; notify decision	criteria count(), record update, ifnull
DS-05	Volunteer Requirement → Validations on form submission	Validate slots; set Filled when full	if, alert, field update
DS-06	Volunteer Registration → Validations on form submission	Accept only college e-mail addresses	endsWith(), alert
DS-07	Event Registration → Successful submission (Edited)	Mark approved applications Completed when event completes	nested for each, record update
DS-08	Schedule: daily 07:00 AM	Remind approved volunteers the day before the event	addDay(), nested fetch, function call

Code 5.1: Application validation (DS-01)

```
// DS-01 | Form: Volunteer_Application | Event: Validations on form
submission (Created)
// Purpose: Block duplicate or invalid applications and initialise the
status
req = Volunteer_Requirement[ID == input.Volunteer_Requirement];
evt = Event_Registration[ID == req.Event];
dup = Volunteer_Application[Volunteer == input.Volunteer &&
Volunteer_Requirement == input.Volunteer_Requirement].count();
if(dup > 0)
{
    alert "You have already applied for this volunteer role.";
    cancel submit;
}
if(req.Requirement_Status != "Open" || req.Slots_Filled >=
req.Slots_Required)
{
    alert "All slots for this role are filled. Please choose another
role.";
    cancel submit;
}
if(evt.Event_Date < zoho.currentdate)
{
    alert "This event is already over.";
    cancel submit;
}
input.Application_Status = "Pending";
input.Application_Date = zoho.currentdate;
```

Code 5.2: Slot recount and decision notification (DS-04)

```
// DS-04 | Form: Volunteer_Application | Event: Successful form
submission (Edited)
// Purpose: Recount filled slots and inform the volunteer of the
decision
req = Volunteer_Requirement[ID == input.Volunteer_Requirement];
filled = Volunteer_Application[Volunteer_Requirement ==
input.Volunteer_Requirement && (Application_Status == "Approved" ||
Application_Status == "Completed")].count();
req.Slots_Filled = filled;
if(filled >= req.Slots_Required)
{
    req.Requirement_Status = "Filled";
}
else if(req.Requirement_Status == "Filled")
{
    req.Requirement_Status = "Open";
}
if(input.Application_Status == "Approved" || input.Application_Status
== "Rejected")
{
    thisapp.notifyVolunteer(input.Volunteer, "Application " +
input.Application_Status + ": " + req.Role_Name, "Your application " +
input.Application_ID + " has been " + input.Application_Status + ".
Remarks: " + ifnull(input.Organizer_Remarks, "-") + ". Reporting time:
" + req.Reporting_Time + ".");
}
```

DS-01 fetches the selected requirement and its event, counts existing applications by the same volunteer for the same role, and cancels the submission with a clear message if the volunteer has already applied, the role is full or closed, or the event is over. Otherwise it sets the status to Pending and the application date to today. In testing, a second application for the same role and an application for a full role were both blocked (TC-06, TC-07).

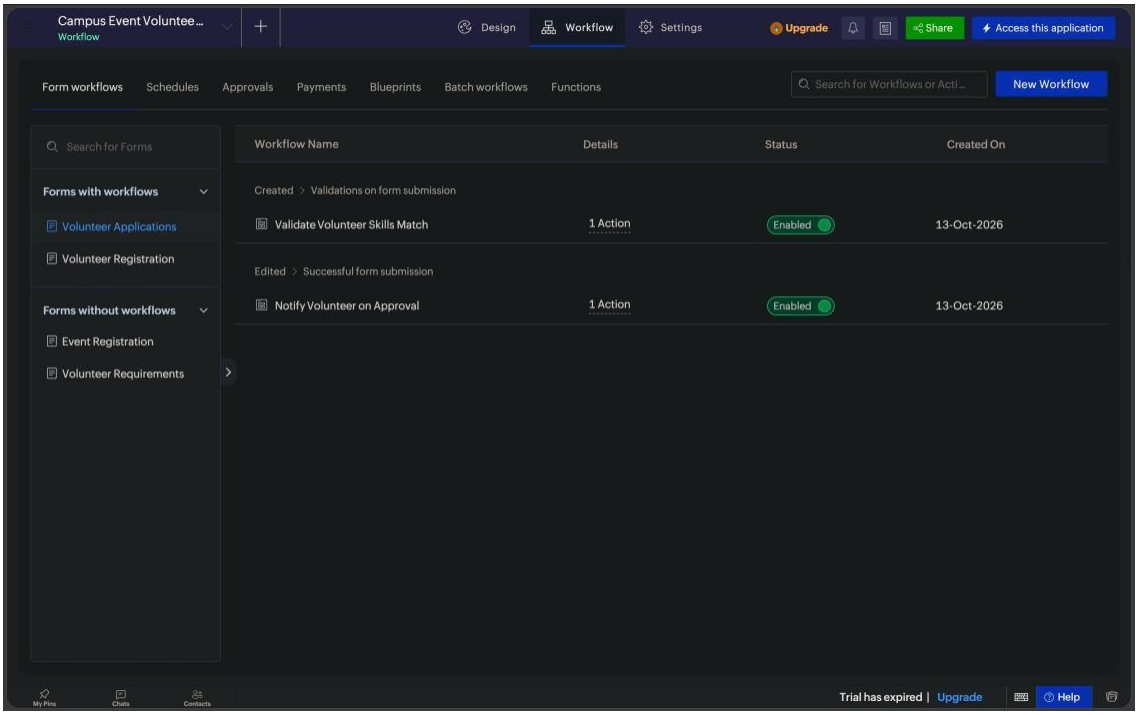
DS-04 runs whenever an organiser edits an application. Instead of adding or subtracting one, it recounts the Approved and Completed applications for the role, so the slot count stays correct even when a decision is changed. It marks the requirement Filled when the count reaches the slots required (and re-opens it if a slot is freed), and e-mails the volunteer for Approved or Rejected decisions with the organiser's remarks. The three Deluge code listings included in this report are reproduced in Appendix C [9].

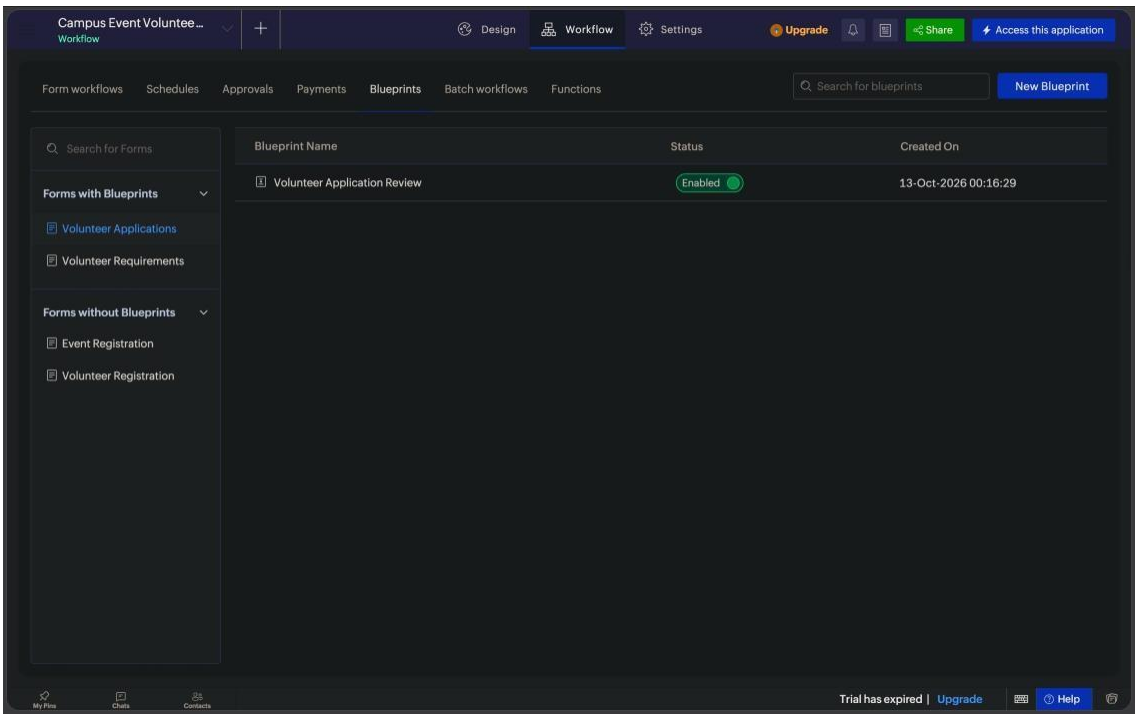
5.6 Workflows and Automation

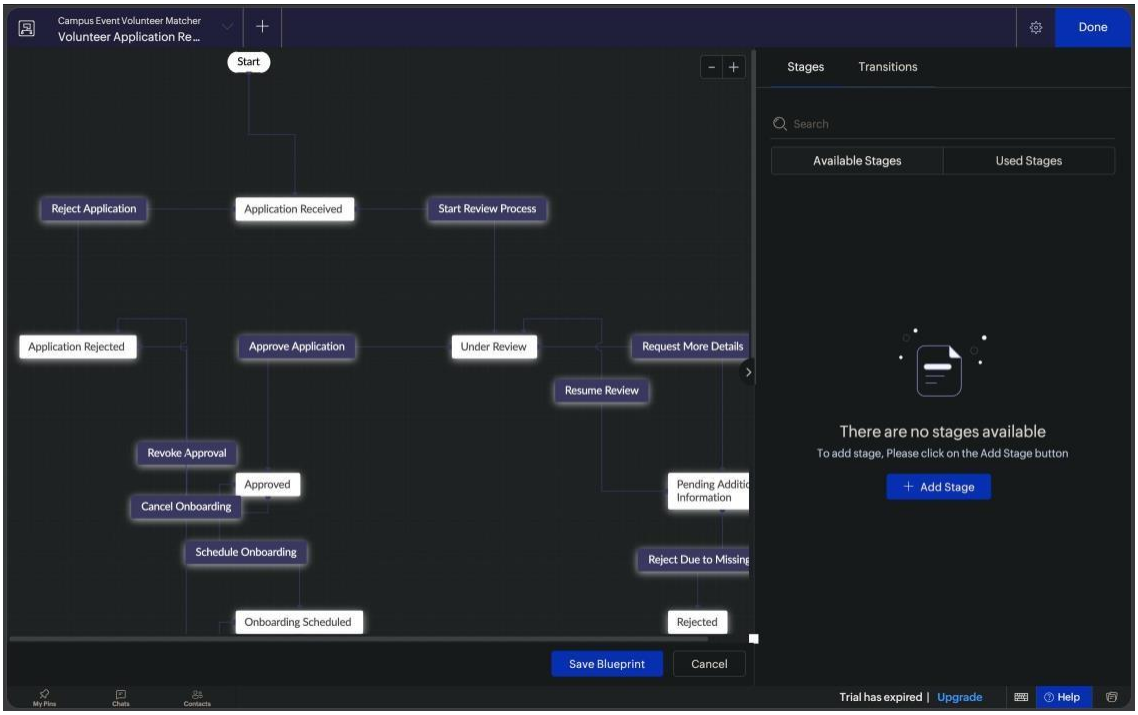
Table 5.3: Workflows Implemented

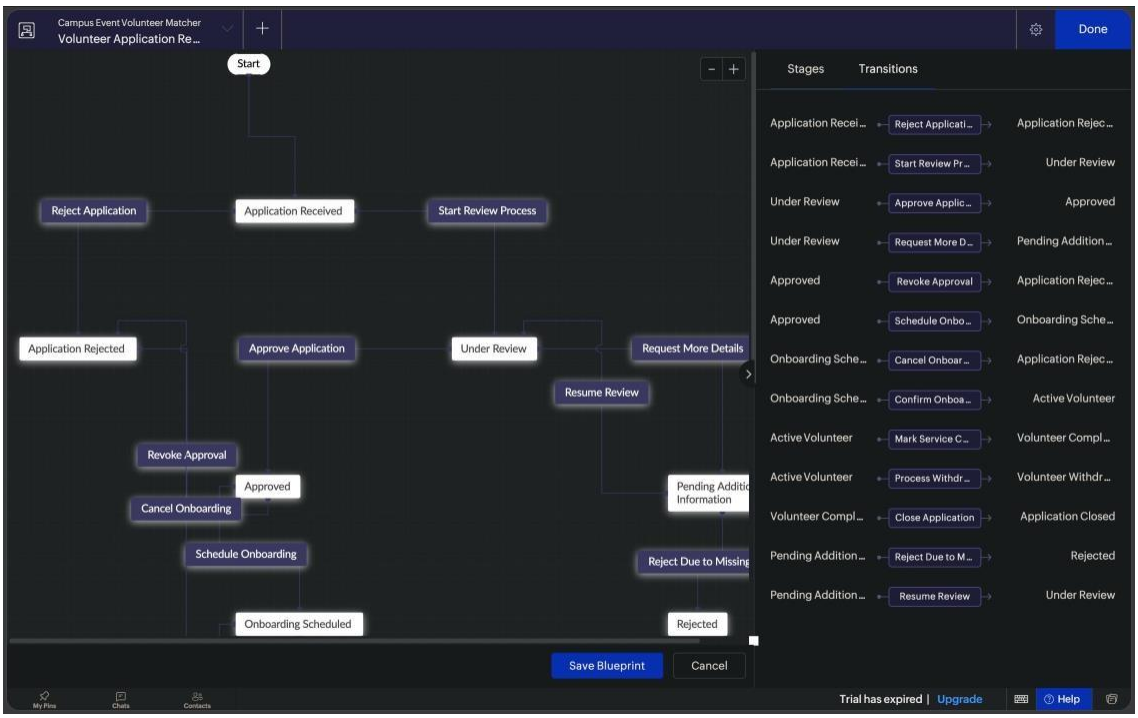
Workflow Name	Type	Trigger / Event	Condition	Action(s)	Script Ref.
Application Validation	Form Workflow	Validations on form submission (Created)	—	Block invalid applications; set Pending	DS-01
Application Acknowledgement	Form Workflow	Successful form submission (Created)	—	E-mail the volunteer	DS-03, DS-02
Decision and Slot Update	Form Workflow	Successful form submission (Edited)	—	Recount slots; notify decision	DS-04
Requirement Validation	Form Workflow	Validations on form submission	—	Validate slots; set Filled	DS-05
Volunteer E-mail Check	Form Workflow	Validations on form submission	—	Require college e-mail	DS-06
Event Completion	Form Workflow	Successful form submission (Edited)	Event Status = Completed	Applications → Completed; requirements Closed	DS-07
Volunteer Reminder	Schedule	Every day, 07:00 AM	Event date = tomorrow	E-mail approved volunteers	DS-08

Application decisions are made by organisers through the Application Status field and Organizer Remarks rather than a formal approval process, because each event's organiser is the only decision-maker. A Zoho Creator approval process and a Blueprint for the application lifecycle are listed as enhancements (Section 9.2). Batch workflows were not required.


Campus Event Voluntee...
Workflow
Design
Workflow
Settings
Upgrade
Share
Access this application
Form workflows
Schedules
Approvals
Payments
Blueprints
Batch workflows
Functions
Search for Workflows or Acti...
New Workflow
Search for Forms
Forms with workflows
Volunteer Applications
Volunteer Registration
Forms without workflows
Event Registration
Volunteer Requirements
Workflow Name
Details
Status
Created On
Created > Validations on form submission
Validate Volunteer Skills Match
1 Action
Enabled
13-Oct-2026
Edited > Successful form submission
Notify Volunteer on Approval
1 Action
Enabled
13-Oct-2026
My Pins
Chats
Contacts
Trial has expired | Upgrade
Help


Campus Event Voluntee...
Workflow
Design
Workflow
Settings
Upgrade
Share
Access this application
Form workflows
Schedules
Approvals
Payments
Blueprints
Batch workflows
Functions
Search for blueprints
New Blueprint
Search for Forms
Forms with Blueprints
Volunteer Applications
Volunteer Requirements
Forms without Blueprints
Event Registration
Volunteer Registration
Blueprint Name
Status
Created On
Volunteer Application Review
Enabled
13-Oct-2026 00:16:29
My Pins
Chats
Contacts
Trial has expired | Upgrade
Help


Campus Event Volunteer Matcher
Volunteer Application Re...
Done
Start
Stages
Transitions
Search
Available Stages
Used Stages
Reject Application
Application Received
Start Review Process
Application Rejected
Approve Application
Under Review
Request More Details
Resume Review
There are no stages available
To add stage, Please click on the Add Stage button
Add Stage
Revoke Approval
Approved
Cancel Onboarding
Pending Additi
Information
Schedule Onboarding
Reject Due to Missing
Onboarding Scheduled
Rejected
Save Blueprint
Cancel
Trial has expired | Upgrade
Help


Campus Event Volunteer Matcher
Volunteer Application Re...
Done
Start
Stages
Transitions
Application Recei...
Reject Applicati...
Application Rejec...
Application Recei...
Start Review Pr...
Under Review
Under Review
Approve Applic...
Approved
Under Review
Request More D...
Pending Addition...
Approved
Revoke Approval
Application Rejec...
Approved
Schedule Onbo...
Onboarding Sche...
Onboarding Sche...
Cancel Onboar...
Application Rejec...
Onboarding Sche...
Confirm Onboa...
Active Volunteer
Active Volunteer
Mark Service C...
Volunteer Compl...
Active Volunteer
Process Withdr...
Volunteer Withdr...
Volunteer Compl...
Close Application
Application Closed
Pending Addition...
Reject Due to M...
Rejected
Pending Addition...
Resume Review
Under Review
Reject Application
Application Received
Start Review Process
Application Rejected
Approve Application
Under Review
Request More Details
Resume Review
Revoke Approval
Approved
Cancel Onboarding
Pending Additi
Information
Schedule Onboarding
Reject Due to Missing
Onboarding Scheduled
Rejected
Save Blueprint
Cancel
Trial has expired | Upgrade
Help

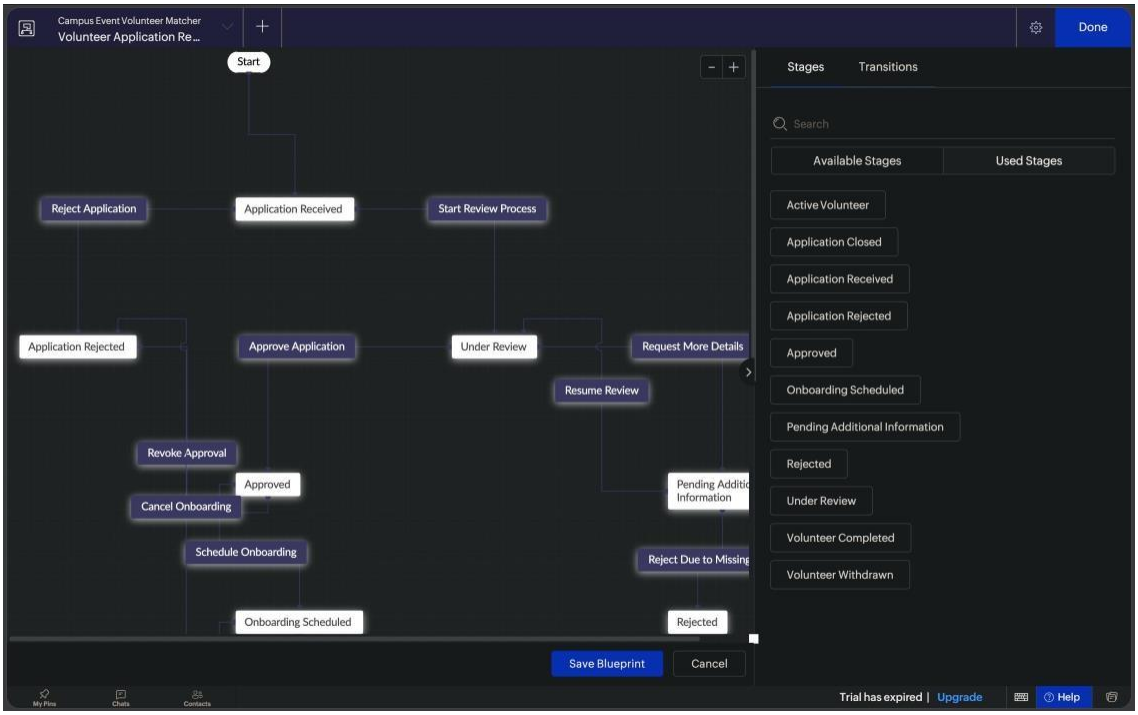


Figure 5.5: Form Workflows and Reminder Schedule Configuration

5.7 Integrations

The application uses Zoho Mail through the Deluge sendmail task. All volunteer e-mails go through one function, notifyVolunteer (Code 5.3), so the greeting and signature are consistent and no credentials are stored in scripts. No third-party API was required.

Table 5.4: Integrations Implemented

Integration	Service / Endpoint	Method	Authentication	Data Exchanged / Purpose
Volunteer E-mails	Zoho Mail (sendmail task)	Deluge sendmail through the notifyVolunteer function	Application owner account (zoho.adminuserid); no stored credentials	Acknowledgements, decisions with remarks and event reminders
External REST API / Webhook	—	Not implemented in this version	—	See Sections 5.10 and 9.2

Code 5.3: Reusable notification function notifyVolunteer (DS-02)

```
// DS-02 | Function: notifyVolunteer | Arguments: volunteerId (int),
subj (string), body (string)
// Purpose: Single place for all e-mails to volunteers
void notifyVolunteer(int volunteerId, string subj, string body)
{
    vol = Volunteer_Registration[ID == volunteerId];
    sendmail
    [
        from :zoho.adminuserid
        to :vol.Email
        subject :subj
        message :"Dear " + vol.Volunteer_Name.first_name + ",<br><br>"
+ body + "<br><br>Regards,<br>Event Volunteer Team"
    ]
}
```

5.8 User Management and Role-Based Access

The permission matrix of Table 4.6 was implemented through **Users and Permissions**. Users were assigned the roles Administrator, Event Organizer, Student Volunteer and Faculty Advisor, each with a permission profile granting the rights in the matrix.

Students see only their own records: the My Applications report uses the criterion *Volunteer.Email == zoho.loginuserid*. Organisers can change only Application Status and Organizer Remarks on applications, and the Faculty Advisor has view-only access to reports and the dashboard.

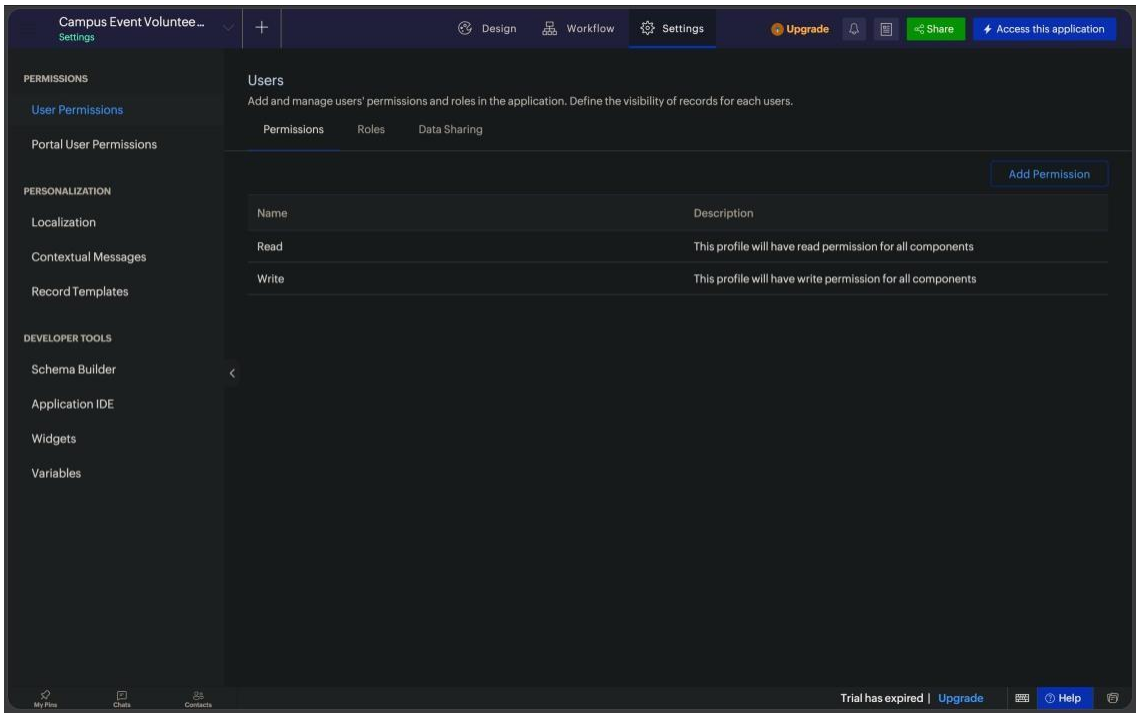


Figure 5.6: Roles and Permission Profiles Configuration

5.9 Mobile Access

The application is available on the Zoho Creator mobile app from the same configuration, so students can register, apply and track their applications from their phones, and organisers can review applications on the go. No separate portal is configured.

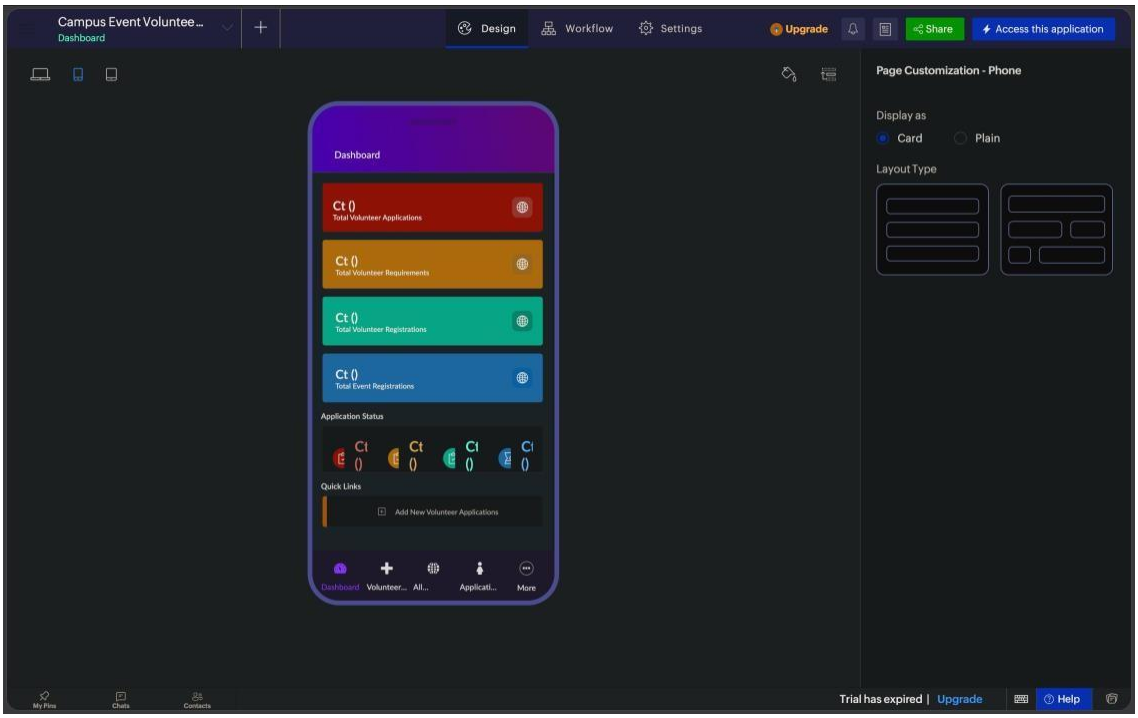


Figure 5.7: Application on the Zoho Creator Mobile App

CHAPTER 6
TESTING AND DEBUGGING

6.1 Testing Strategy

Testing was carried out at four levels:

- **Unit testing** of each form and script with sample events, roles, volunteers and applications.
- **Integration testing** of the application flow: validation, acknowledgement, decision e-mail, slot recount, event completion and the reminder schedule (run manually).
- **Role-based testing** by logging in as each role and checking visible modules and actions.
- **User acceptance testing** with student volunteers and an organiser (Section 6.5).

Each test case is traced to a functional requirement in Table 3.2.

6.2 Test Cases

Table 6.1: Test Cases

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-01	FR-01	Register an event	Symposium with date and venue	EVT-001 created, Upcoming	As expected	Pass
TC-02	FR-02	Register a volunteer	College e-mail, skills	VOL-001 created	As expected	Pass
TC-03	FR-02	Register with personal e-mail	abc@gmail.com	Alert: use college e-mail	As expected	Pass
TC-04	FR-03	Add requirement	Registration Desk, 5 slots	REQ-001 Open, filled 0	As expected	Pass
TC-05	FR-03	Requirement with 0 slots	Slots Required = 0	Alert; not saved	As expected	Pass
TC-06	FR-05	Duplicate application	Same volunteer, same role	Alert “already applied”	As expected	Pass
TC-07	FR-05	Apply for full role	Role with all slots filled	Alert “slots filled”	As expected	Pass
TC-08	FR-05	Apply for past event	Event date yesterday	Alert “event over”	As expected	Pass
TC-09	FR-06	Valid application	Reason entered	Status Pending; acknowledgement e-mail	As expected	Pass

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-10	FR-07, FR-08	Approve application	Status Approved + remarks	Slots filled +1; e-mail sent	As expected	Pass
TC-11	FR-08	Fill last slot	Approve 5th of 5	Requirement → Filled	As expected	Pass
TC-12	FR-08	Change approved to rejected	Edit status	Slots filled −1; requirement re-opened	As expected	Pass
TC-13	FR-09	Complete event	Event Status Completed	Approved → Completed; requirements Closed	As expected	Pass
TC-14	FR-10	Reminder schedule	Event tomorrow; run schedule	Approved volunteers e-mailed	As expected	Pass
TC-15	FR-11	Dashboard counts	Sample data	Totals and status counts match reports	As expected	Pass
TC-16	FR-12	Student views others' applications	Log in as Student Volunteer	Only own applications visible	As expected	Pass

6.3 Debugging Deluge Scripts

Scripts were debugged with **info** statements that print fetched values (for example, the duplicate count in DS-01 and the recounted slots in DS-04), the **Execute** option of the function editor for notifyVolunteer, the workflow and schedule logs, and the error messages shown on form submission.

A representative bug (BUG-01 in Table 6.2): when an organiser changed an approved application to rejected, the requirement still showed the old number of filled slots. The first version of DS-04 only added one on approval. Replacing the increment with a recount of Approved and Completed applications fixed both directions.

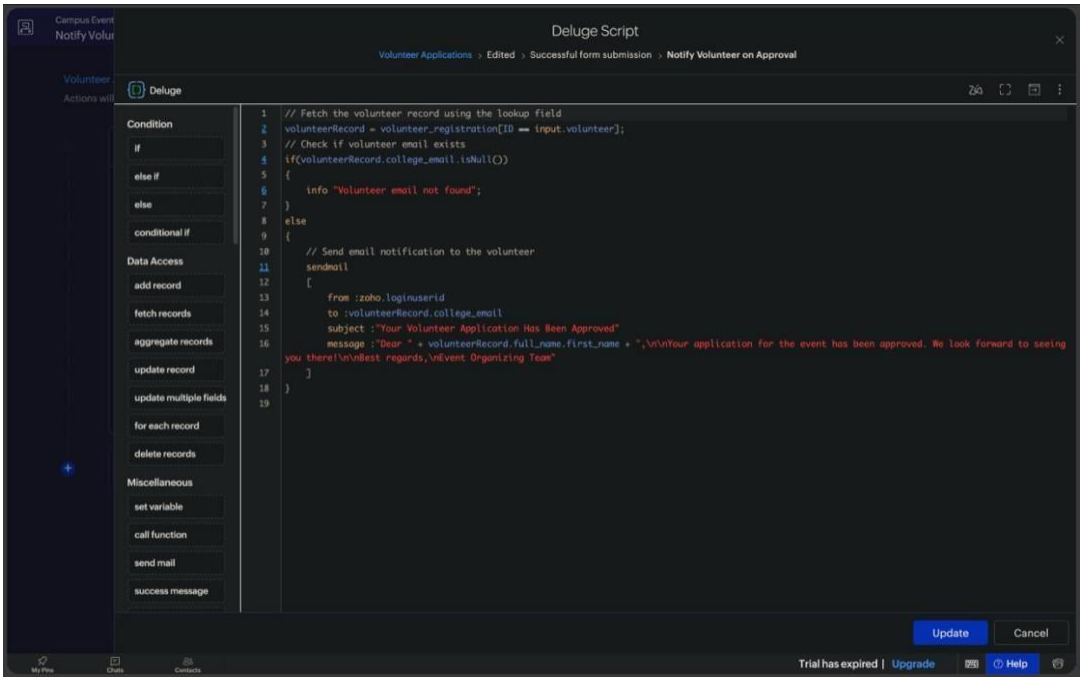


Figure 6.1: Deluge info Output while Debugging the Slot Recount

6.4 Defect Log

Table 6.2: Defect Log

Bug ID	Description	Root Cause	Fix Applied	Status
BUG-01	Slots Filled not reduced when an approval was reversed	DS-04 incremented the count only on approval	Recount Approved and Completed applications	Closed
BUG-02	Requirement stayed Filled after a slot was freed	No re-open logic	Re-open in DS-04 when count < slots	Closed
BUG-03	Decision e-mail showed “null” when remarks were empty	Empty remarks concatenated directly	Wrapped in ifnull()	Closed
BUG-04	Students could apply for roles of completed events	Lookup showed all requirements	Filter lookup to Open; event-date check in DS-01	Closed

CHAPTER 7
DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

The application was developed, tested and demonstrated in a single Zoho Creator application workspace on the free trial edition (India data centre). Separate Stage and Production environments were not configured; the live application URL is used for the demonstration.

Changes were controlled by testing each form, script and schedule after it was built, marking sample records clearly and removing them before live use, and exporting data before major changes. On a paid edition the application would be promoted through Development → Stage → Production with version notes and a rollback point.

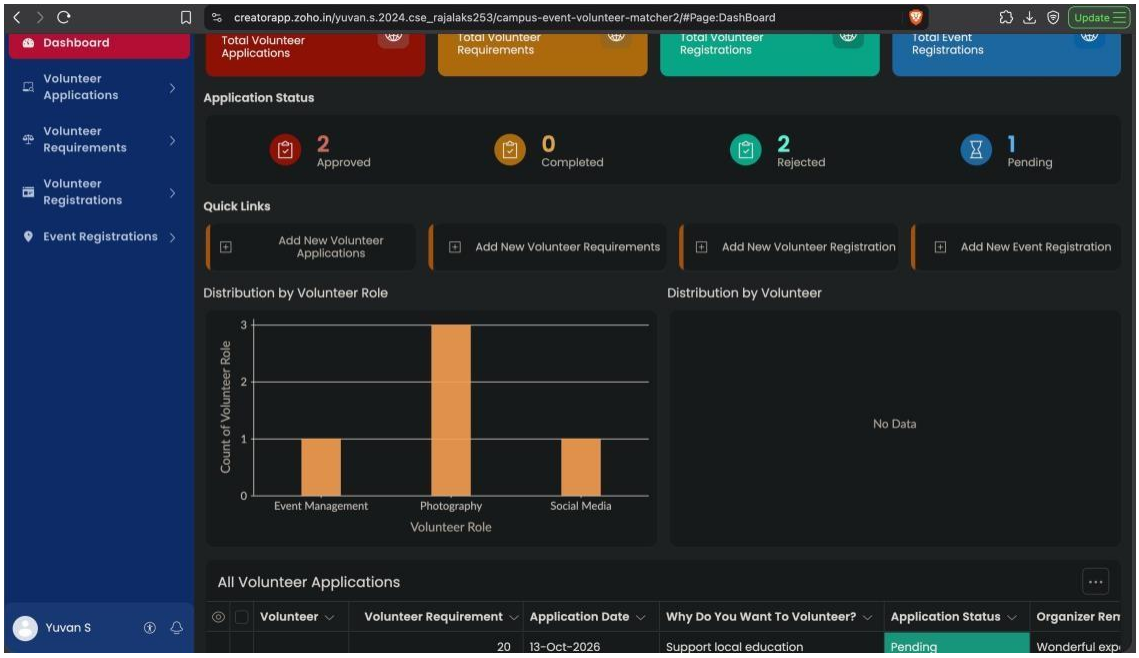


Figure 7.1: Live Application Accessed through the Published URL

Table 7.1: Deployment Checklist

S.No.	Checklist Item	Done (✓)
1	All test cases executed in the application workspace	✓
2	Sample/test data removed or isolated	✓
3	Roles and permission profiles configured	✓
4	Connections authorised (none required)	N/A
5	Schedules and workflows enabled	✓

S.No.	Checklist Item	Done (✓)
6	Application URL verified	✓
7	Change notes recorded	✓

7.2 Security Features

Table 7.3: Security Controls Implemented

Security Control	Where Applied	Threat Mitigated
Role-based permission profiles	All forms, reports and the dashboard	Unauthorised access
Record-level criteria on zoho.loginuserid	My Applications report	Students viewing others' data
Status and remarks editable only by organisers	Volunteer Application	Students approving themselves
College e-mail validation	Volunteer Registration (DS-06)	Outsiders registering
No hard-coded credentials; sendmail from zoho.adminuserid	DS-02	Credential leakage
Audit trail	All forms	Untraceable changes

7.3 Governance and Administration

Governance was established through the following practices:

- **Audit trail** records changes to application status and remarks.
- **Naming conventions:** forms named after the module, link names in Title_Case with underscores, and scripts headed with their DS number and purpose.
- **Data housekeeping:** completed events are closed by DS-07, and data is exported at the end of each semester.
- **User management:** graduated students are removed from users each year while their records are kept.
- **Ownership:** the administrator owns the application and approves changes to roles and workflows.

CHAPTER 8
RESULTS AND DISCUSSION

8.1 Application Output

The application was demonstrated with one event. The organiser registered a symposium and posted requirements for Registration Desk, Hospitality and Technical Support. Students registered as volunteers and applied for roles; each received an acknowledgement (Figure 8.1). The organiser approved and rejected applications with remarks, the filled-slot counts updated and the Registration Desk requirement became Filled (Figure 8.2). The dashboard then showed the updated totals, status counts and role-wise distribution (Figure 8.3).

8.2 Achievement of Objectives

Table 8.1: Mapping of Objectives to Outcomes

Objective (Sec. 1.3)	Implemented Through	Outcome / Evidence
Design the data model and forms	Sections 4.2, 4.3, 5.2	Four linked forms; invalid input rejected (TC-01 to TC-05)
Automate validation, slots, notifications and reminders	Sections 5.5, 5.6	Duplicates and full roles blocked; slots and e-mails automatic (TC-06 to TC-14)
Provide a role-based dashboard and reports	Sections 5.3, 5.4	Eight count panels and role chart (TC-15)
Integrate with Zoho Mail	Section 5.7	Acknowledgement, decision and reminder e-mails delivered
Deploy securely with governance	Sections 5.8, 7.1–7.4	Four roles; students see only own records (TC-16)

8.3 Comparison with the Existing System

Table 8.2: Existing System versus Proposed Application

Parameter	Existing System	Proposed Application	Improvement
Collecting applications	Messages and forms copied into sheets	Applications in one form	No manual copying
Duplicate and full-role checks	Manual	Automatic (DS-01)	Errors prevented
Role allocation visibility	Not available	Live slot counts and role chart	Balanced allocation

Parameter	Existing System	Proposed Application	Improvement
Communicating decisions	Individual messages	Automatic e-mails with remarks	Faster, consistent
Records after the event	None	Completed applications per event	Participation history kept

8.4 Limitations of the Low-Code Approach

Building the application on Zoho Creator also showed the limits of the low-code approach:

- **Derived counts:** Creator has no built-in roll-up field, so filled slots had to be maintained by a script (DS-04), which must also handle reversed decisions.
- **Script-triggered updates:** records changed by scripts (for example, DS-07) do not fire form workflows, so related updates had to be written in the same script.
- **UI customisation:** dashboard layouts are limited to Creator's components; drag-and-drop allocation would need a widget.
- **E-mail limits:** daily sendmail limits on the free trial restrict reminders for very large events.

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

The Campus Event Volunteer Matcher was built on Zoho Creator to replace the message-and-spreadsheet process for managing campus event volunteers. Its four forms cover events, volunteers, role requirements and applications; eight Deluge scripts validate applications, maintain slot counts, notify volunteers and send reminders; and a dashboard shows totals, status counts and the distribution of volunteers across roles.

All objectives were met (Table 8.1): manual sorting and messaging were eliminated, roles are filled evenly, and organisers and volunteers have a transparent, shared view of every application.

9.2 Future Enhancements

- **Skill-based matching with Zia** to suggest suitable volunteers for each role.
- **Approval process and Blueprint** for multi-level approval and a formal application lifecycle.
- **QR-based attendance and volunteer certificates** generated automatically for completed applications.
- **Integration with the college activity-points system** through the Creator REST API.

REFERENCES

[1] W. Rosam and O. Kalnichevski, *Low-Code Development: A Practical Guide*. New Delhi, India: BPB Publications.

[2] Zoho Corporation, "Zoho Creator Help Documentation." [Online]. Available: https://www.zoho.com/creator/help/ (Accessed: –).

[3] Gartner, Inc., "Magic Quadrant for Enterprise Low-Code Application Platforms," *Gartner Research*, 2024.

[4] A. Sahay, A. Indamutsa, D. Di Ruscio and A. Pierantonio, "Supporting the understanding and comparison of low-code development platforms," in *Proc. 46th Euromicro Conf. Software Engineering and Advanced Applications (SEAA)*, 2020, pp. 171–178.

[5] R. Sanchis, Ó. García-Perales, F. Fraile and R. Poler, "Low-code as enabler of digital transformation in manufacturing industry," *Applied Sciences*, vol. 10, no. 1, p. 12, 2020.

[6] M. Hirzel, "Low-code programming models," *Communications of the ACM*, vol. 66, no. 10, pp. 76–85, 2023.

[7] Zoho Corporation, "Zoho Backstage – Event Management Software." [Online]. Available: https://www.zoho.com/backstage/ (Accessed: –).

[8] Zoho Corporation, "Zoho Creator Learning Centre." [Online]. Available: https://www.zoho.com/creator/resources/ (Accessed: –).

[9] Zoho Corporation, "Deluge Script Reference Guide." [Online]. Available: https://www.zoho.com/deluge/ (Accessed: –).

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST

Table A.1: Campus Event Volunteer Matcher – Feature Coverage

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
I	Application created and menu organised	Y	Five modules: Dashboard and four registration/application modules	5.1	Fig. 5.1
II	Forms and field configuration	Y	Four forms with specified fields, lookups and validations	4.2–4.3, 5.2	Tables 4.1–4.5
II	Lookup relationships between forms	Y	Event, Volunteer and Volunteer Requirement lookups	4.2, 5.2	Fig. 4.2; Table 4.1
II	List reports and application filtering/grouping	Y	List reports for forms; My Applications and All Volunteer Applications	5.3	Table 5.1; Fig. 5.3
II	Dashboard and chart	Y	Dashboard with total panels, status panels and volunteer-role chart	5.4	Fig. 5.4
II	Deluge scripts shown in the report	Y	DS-01 validation, DS-04 slot recount/decision notice, DS-02 notification function	5.5, 5.7	Code 5.1–5.3; Table 5.2
II	Form workflows and scheduled reminder	Y	Six form workflows and daily volunteer reminder schedule as listed	5.6	Table 5.3; Fig. 5.5
III	Zoho Mail integration	Y	Volunteer acknowledgement, decision and reminder e-mails through sendmail	5.7	Table 5.4; Code 5.3
III	User roles and permission profiles	Y	Administrator, Event Organizer, Student Volunteer and Faculty Advisor	4.6, 5.8	Table 4.6; Fig. 5.6
III	Record-level access for student applications	Y	My Applications report filtered to the signed-in volunteer	5.3, 5.8	Table 5.1; Table 4.6

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
IV	Testing and debugging evidence	Y	Test cases, defect log and Deluge debugging output	Chapter 6	Tables 6.1, 6.2; Fig. 6.1
IV	Deployment approach	Y	Single Creator workspace; separate Stage/Production environments not configured	7.1	Table 7.1; Fig. 7.1
IV	Security controls	Y	Role-based permissions and security controls documented	7.2	Table 7.3
IV	Governance and administration	Y	Audit trail, naming conventions, data housekeeping and ownership	7.3	—
—	Mobile app access	Y	Application accessed through Zoho Creator mobile app	5.9	Fig. 5.7

Features documented in the Campus Event Volunteer Matcher report: 15

APPENDIX B – COURSE OUTCOME MAPPING

Table B.1: Mapping of Course Outcomes to Report Chapters

CO	Course Outcome Statement	Evidenced In	Self-Assessed Level	Examiner Level
CO1	Explain the principles, architecture, and business value of low-code application development using Zoho Creator.	Ch. 1, 2	Chapters 1–2	
CO2	Develop business applications using Zoho Creator forms, reports, workflows, and Deluge scripting.	Ch. 4, 5 (5.1–5.6), 6	Chapters 4–6	
CO3	Integrate Zoho Creator applications with Zoho services and external systems using APIs, connections, and role-based security.	Ch. 4.6, 5.7–5.9	Sections 4.6, 5.7–5.8	
CO4	Deploy, secure, and manage Zoho Creator applications across development and production environments following governance best practices.	Ch. 7	Chapter 7	
CO5	Evaluate low-code development strategies, AI-assisted application development, and the transition from citizen developer to professional developer.	Ch. 5.9, 5.10, 8.4, 8.5, 9.3	Sections 5.5, 8.4, 9.2	

APPENDIX C – COMPLETE DELUGE SOURCE CODE

C.1 Volunteer_Application – Application Validation (DS-01)

```
// DS-01 | Form: Volunteer_Application | Event: Validations on form
submission (Created)
// Purpose: Block duplicate or invalid applications and initialise the
status
req = Volunteer_Requirement[ID == input.Volunteer_Requirement];
evt = Event_Registration[ID == req.Event];
dup = Volunteer_Application[Volunteer == input.Volunteer &&
Volunteer_Requirement == input.Volunteer_Requirement].count();
if(dup > 0)
{
    alert "You have already applied for this volunteer role.";
    cancel submit;
}
if(req.Requirement_Status != "Open" || req.Slots_Filled >=
req.Slots_Required)
{
    alert "All slots for this role are filled. Please choose another
role.";
    cancel submit;
}
if(evt.Event_Date < zoho.currentdate)
{
    alert "This event is already over.";
    cancel submit;
}
input.Application_Status = "Pending";
input.Application_Date = zoho.currentdate;
```

C.2 Volunteer_Application – Slot Recount and Decision Notification (DS-04)

```
// DS-04 | Form: Volunteer_Application | Event: Successful form
submission (Edited)
// Purpose: Recount filled slots and inform the volunteer of the
decision
req = Volunteer_Requirement[ID == input.Volunteer_Requirement];
filled = Volunteer_Application[Volunteer_Requirement ==
input.Volunteer_Requirement && (Application_Status == "Approved" ||
Application_Status == "Completed")].count();
req.Slots_Filled = filled;
if(filled >= req.Slots_Required)
{
    req.Requirement_Status = "Filled";
}
else if(req.Requirement_Status == "Filled")
{
    req.Requirement_Status = "Open";
}
if(input.Application_Status == "Approved" || input.Application_Status
== "Rejected")
{
    thisapp.notifyVolunteer(input.Volunteer, "Application " +
input.Application_Status + ": " + req.Role_Name, "Your application " +
input.Application_ID + " has been " + input.Application_Status + ".
Remarks: " + ifnull(input.Organizer_Remarks, "-") + ". Reporting time:
" + req.Reporting_Time + ".");
}
```

C.3 Function – notifyVolunteer (DS-02)

```
// DS-02 | Function: notifyVolunteer | Arguments: volunteerId (int),
subj (string), body (string)
// Purpose: Single place for all e-mails to volunteers
void notifyVolunteer(int volunteerId, string subj, string body)
{
    vol = Volunteer_Registration[ID == volunteerId];
    sendmail
    [
        from :zoho.adminuserid
        to :vol.Email
        subject :subj
        message :"Dear " + vol.Volunteer_Name.first_name + ",<br><br>"
+ body + "<br><br>Regards,<br>Event Volunteer Team"
    ]
}
```

APPENDIX D – USER MANUAL

D.1 Accessing the Application

Web: open the Campus Event Volunteer Matcher application URL provided by the administrator and sign in with the authorised Zoho account. Mobile: install the Zoho Creator mobile app, sign in with the same account and open Campus Event Volunteer Matcher. Available modules depend on the assigned role. The report states that no separate portal is configured.

D.2 Instructions for the Event Organizer

- Open **Event Registrations → Add** to register the event with date, venue and organiser details.
- Open **Volunteer Requirements → Add** for each role, with slots required and reporting time.
- Open **Volunteer Applications**, filter Pending, and set each application to Approved or Rejected with remarks.
- After the event, set the event's status to Completed; approved applications become Completed automatically.

D.3 Instructions for a Student Volunteer

- Open **Volunteer Registrations → Add** once, using your college e-mail.
- Open **Volunteer Applications → Add**, choose an open role and write your reason for volunteering.
- Track your applications in **My Applications** and check your e-mail for decisions and reminders.

D.4 Instructions for the Faculty Advisor

- Open the **Dashboard** to see totals, status counts and the role-wise distribution of volunteers.
- Use the list reports to review events, requirements and applications (view only).

APPENDIX E – EVALUATION RUBRIC

(For Examiner Use Only)

Student Name	THIRUMALA SRI P	Register No.	2116240701569
Project Title	Campus Event Volunteer Matcher	Date	–

Table E.1: Evaluation Rubric for the Individual Project

S.No.	Criterion	Indicators	Max. Marks	Marks Awarded
1	Problem Definition & Requirement Analysis	Clarity of problem, measurable objectives, complete FR/NFR, use cases	10	
2	System Design	Architecture, ER/data model, form specifications, workflow design, permission matrix	10	
3	Forms, Reports & Pages	Variety of field types, lookups/subforms, report types, UI Builder dashboards	15	
4	Deluge Scripting & Workflows	Correctness and depth of scripts; form, schedule, approval, Blueprint, batch workflows	15	
5	Integration, RBAC & AI	Zoho/external integrations, connections, roles & permissions, Zia/analytics	10	
6	Testing, Deployment, Security & Governance	Test coverage, environments, optimisation, security controls, audit	10	
7	Live Demonstration	End-to-end working of the application across roles; feature checklist verified	15	
8	Project Report	Adherence to template and format, clarity, figures, references	5	
9	Viva-Voce	Understanding of the platform, design decisions and low-code concepts	10	
	TOTAL		**100**	

Table E.2: Performance Level Descriptors

Level	% of Max. Marks	Descriptor
Excellent	90–100	Feature implemented completely, correctly and beyond the basic requirement; student explains design choices confidently.
Good	70–89	Feature implemented correctly with minor gaps; explanation adequate.
Satisfactory	50–69	Feature partially implemented or works only for the main path; limited explanation.
Needs Improvement	< 50	Feature missing, non-functional or not understood by the student.

Signature of Internal Examiner

Name: _____

Signature of External Examiner

Name: _____

APPENDIX F – VIVA-VOCE RECORD SHEET

(For Examiner Use Only)

Student Name	THIRUMALA SRI P	Register No.	2116240701569
Department / Section	CSE / [Section]	Date of Viva	–
Application URL Verified	Yes / No	Demo Duration	_____min

Table F.1: Viva-Voce Questions and Responses

Q.No.	Question / Topic Asked	Unit	Response (E / G / S / NI)	Remarks
1				
2				
3				
4				
5				
6				
7				
8				

E – Excellent G – Good S – Satisfactory NI – Needs Improvement

Table F.2: Marks Summary

Component	Max. Marks	Marks Awarded
Criteria 1–8 (Appendix E)	90	
Viva-Voce	10	
TOTAL	**100**	

Examiners' Remarks:

_____

_____

INTERNAL EXAMINER

Name: _____

EXTERNAL EXAMINER

Name: _____